# Étude de cas 5 : Maintenance prédictive

## Jalon 2 : Écouter les vibrations

### chapitre 2 - CNN et auto-encodeurs

M2 SDIA - Advanced Artificial Intelligence

Année universitaire 2026–2027

**Objectif du jalon**

Reconnaître le mode de défaillance sur les spectrogrammes et construire un indicateur de santé non supervisé.

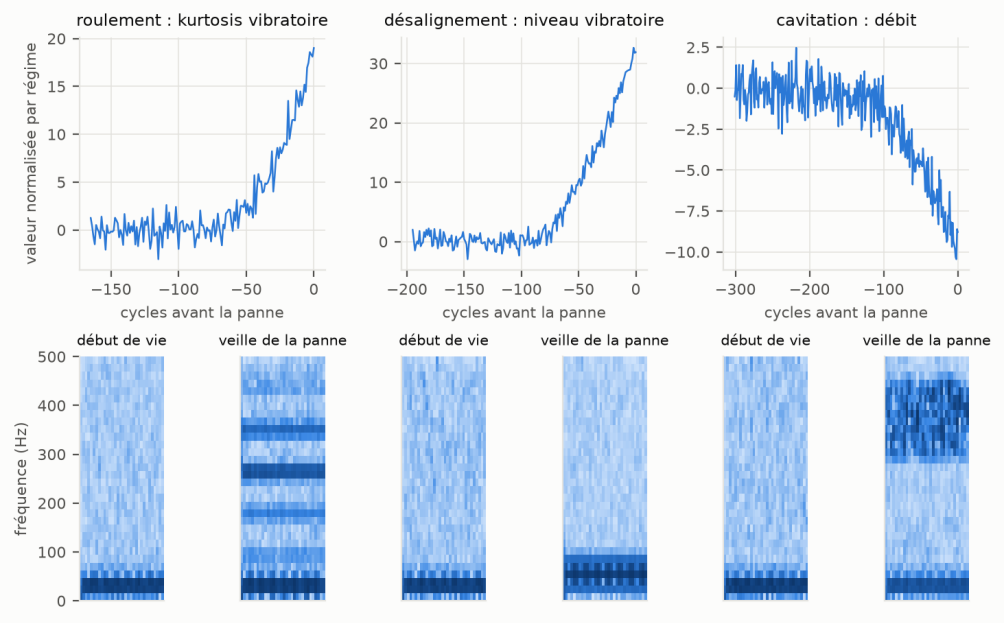
**Jalon autonome :** ce jalon ne suppose pas d'avoir terminé les jalons précédents. Il s'appuie uniquement sur le notebook de démarrage `uc5_demarrage.ipynb`, le dossier `data/` et le code de démarrage de la section 3. Vous pouvez repartir de votre propre travail antérieur, mais rien ne l'exige.
**Environnement :** Python 3, TensorFlow/Keras, scikit-learn ; Google Colab avec GPU recommandé.

## 1 Contexte

Un site minier fictif exploite une flotte de pompes de lavage. Une panne imprévue arrête la ligne de production ; un remplacement trop précoce gaspille une pièce encore en état. Chaque pompe transmet une fois par jour (un *cycle*) huit mesures agrégées (températures, courant, pression, débit, vitesse, niveau et impulsivité des vibrations) ainsi qu'un relevé vibratoire d'une seconde, transformé en spectrogramme. Trois modes de défaillance existent : usure de roulement, désalignement et cavitation. Les capteurs dépendent aussi du **régime de charge** (60, 80 ou 100 %), qui change d'un jour à l'autre.

Un nouveau modèle de pompe (type B, deux fois plus rapide et plus puissant) vient d'être installé ; seules dix pompes de ce type ont déjà été suivies jusqu'à la panne.

**L'étude de cas en cinq jalons :** un jalon par chapitre du cours, rendu après le chapitre correspondant, puis une synthèse défendue à l'oral en fin de module (note de décision, prédictions sur le test caché, soutenance).




## 2 Données et cibles de ce jalon

| Fichier | Contenu |
|---|---|
| `uc5_flotteA.npz` | type A : 70 pompes suivies jusqu'à la panne (entraînement), 15 (validation), 30 pompes dont l'historique est coupé avant la panne (test) |

Pour chaque cycle : numéro de pompe et de cycle, régime de charge, huit capteurs, spectrogramme $32 \times 32$ (temps sur une seconde en abscisse, fréquences de 0 à 500 Hz en ordonnée) ; pour les historiques complets, la RUL et le mode de défaillance. **Découpage imposé par pompe** : les cycles d'une même pompe ne sont jamais répartis entre deux ensembles ; faites de même pour vos plis de validation croisée. Les données sont simulées.

**Cibles du cahier des charges concernées**

| Exigence | Mesure (ensemble de test) | Cible |
|---|---|---|
| Mode de défaillance | exactitude sur les pompes à 60 cycles ou moins de la panne | $\geq 0{,}85$ |
| Durée de vie restante, type A | RMSE sur la RUL plafonnée à 125 cycles | $\leq 23$ cycles |

La **RUL** est le nombre de cycles avant la panne ; elle est plafonnée à 125 cycles. Une pompe est **signalée** lorsque sa RUL prédite est inférieure à 30 cycles. Le **score asymétrique** (challenge NASA C-MAPSS) vaut, avec $d_i = \hat{y}_i - y_i$ : $s = \sum_i (e^{-d_i/13} - 1)$ si $d_i < 0$ et $s = \sum_i (e^{d_i/10} - 1)$ sinon (fonction `score_nasa` du notebook). Ces cibles décrivent le système *final* : elles ne sont pas exigées à la fin de ce jalon. Mesurez-vous par rapport à elles et expliquez l'écart ; un écart argumenté vaut mieux qu'une cible atteinte sans explication.

# 3 Point de départ

Le notebook de démarrage fournit les capteurs normalisés par régime, les spectrogrammes, les métriques et le **MLP de RUL de référence** (section 3). Le code ci-dessous met les spectrogrammes en tenseurs, propose une règle d'étiquetage (« sain » tant que la RUL dépasse un seuil, mode de défaillance ensuite) et sélectionne les cycles sains pour l'auto-encodeur.

Code de démarrage (fichier `uc5_jalon2_depart.py`)

```
# Étude de cas 5 - Jalon 2 : code de démarrage
# À exécuter à la suite du notebook de démarrage, exécuté en entier.
from scipy.stats import spearmanr

# 1. Spectrogrammes (N, 32, 32, 1) dans [0, 1] : lignes = fréquences
#    (0 à 500 Hz, de bas en haut), colonnes = temps (une seconde)
SPEC = {s: A["spectro_" + s][..., None].astype("float32") / 255.0
        for s in ["train", "val", "test"]}

# 2. Règle d'étiquetage, à justifier et à faire varier : « sain » tant
#    que la RUL dépasse un seuil, mode de défaillance ensuite
def etiquette_mode(D, s, seuil=60):
    # 0 = sain, 1 = roulement, 2 = désalignement, 3 = cavitation
    y = np.where(D["rul_" + s] > seuil, 0, 1 + D["mode_" + s])
    return y.astype("int32")

Y_MODE = {s: etiquette_mode(A, s) for s in ["train", "val"]}
print("effectifs par classe (train) :", np.bincount(Y_MODE["train"]))

# 3. Indicateur de santé non supervisé : cycles sains (RUL > 150, comme
#    pour la normalisation par régime), capteurs normalisés et charge
sains = NA["train"].rul.values > 150
X_sains = NA["train"][CAPTEURS + ["charge"]].values[sains]
print("cycles sains pour l'auto-encodeur :", X_sains.shape)
# après entraînement de l'auto-encodeur (fenêtres de cycles sains) :
# rho = spearmanr(erreur_de_reconstruction, rul).correlation
```

# 4 Travail demandé

Jalon 2 : Écouter les vibrations

**Notions mobilisées :** convolution sur des spectrogrammes, pooling, dimensions et paramètres, auto-encodeur, indicateur de santé, corrélation de rang.

1. Concevoir un CNN qui classe un spectrogramme en « sain », « roulement », « désalignement » ou « cavitation ». Justifier la règle d'étiquetage et la faire varier.
2. Mesurer l'exactitude en fonction de la RUL : à partir de quand le mode devient-il reconnaissable ?
3. Entraîner un auto-encodeur sur des fenêtres de cycles sains ; utiliser l'erreur de reconstruction comme indicateur de santé ; mesurer sa corrélation de rang avec la RUL et le délai de détection de la dégradation.
4. Ajouter l'indicateur de santé ou les sorties du CNN aux entrées du MLP de RUL :

la prévision s'améliore-t-elle ?

**Vérification théorique (à la main, puis vérifiée par le code)**

1. Le relevé est échantillonné à 1000 Hz et le spectrogramme utilise des fenêtres de 62 points. Calculez la résolution en fréquence. Pour une pompe de type A (1480 tr/min), calculez la fréquence de rotation et la fréquence caractéristique de défaut de roulement ($3{,}58 \times f_{rot}$), puis la ligne du spectrogramme où elle apparaît. Même question pour le type B (2960 tr/min).
2. Calculez les dimensions et le nombre de paramètres de votre CNN, puis vérifiez avec `summary()`.

**À justifier dans votre page de décisions**

Le spectrogramme plutôt que le signal brut ; l'intérêt d'un indicateur non supervisé lorsque les pannes sont rares.

# 5 À rendre à la fin du chapitre

**Livrables du jalon 2**

- le notebook du jalon, exécuté et reproductible (graine fixée, versions indiquées) ;
- le journal d'expériences à jour (fonction `journaliser` du notebook de démarrage, avec `jalon=2`), y compris les essais infructueux ;
- une page de décisions : tableau des résultats, choix retenus et justifications, réponses aux vérifications théoriques.

**Protocole :** tous les réglages se font sur la validation ; l'ensemble de test ne sert qu'une fois, à la fin du jalon, pour le modèle retenu. Ce jalon alimente la note de décision finale et la soutenance.